Título:	**Expressões Booleanas e Estrutura Condicional *if-else***
Professor:	Eleandro Maschio
Ano:	2023
Disciplina:	Fundamentos de Programação
Curso:	Sistemas para Internet
Câmpus:	Guarapuava
Instituição:	Universidade Tecnológica Federal do Paraná
Como citar:	Referência



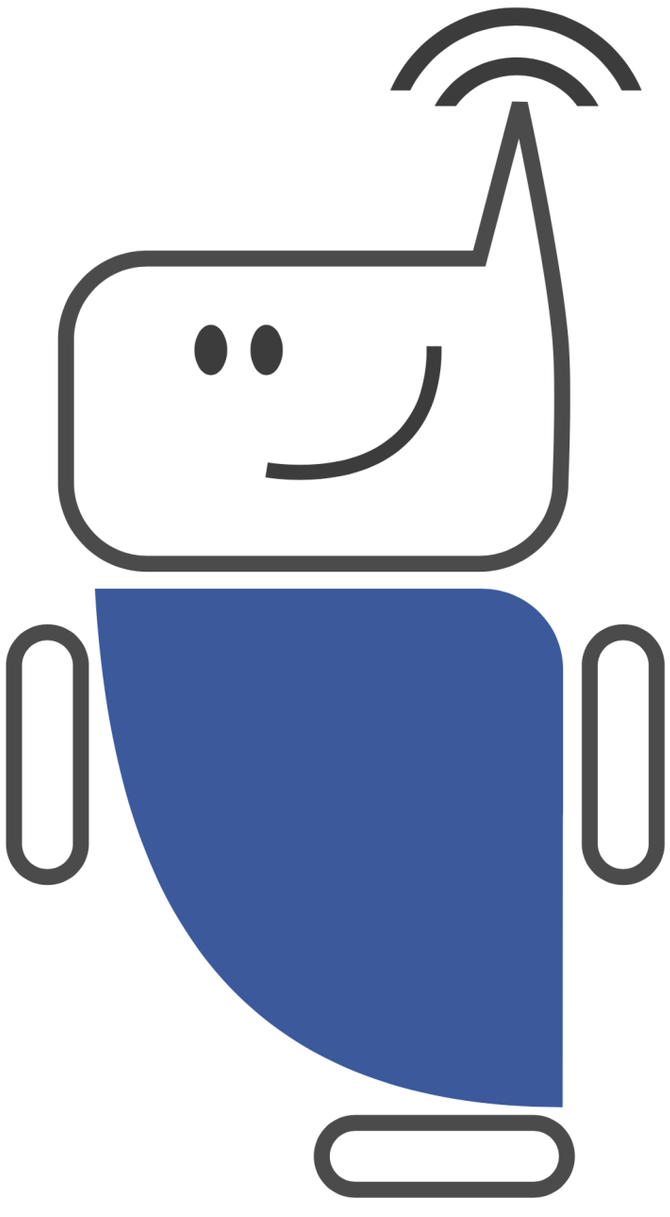


EXPRESSÕES BOOLEANAS E

ESTRUTURA CONDICIONAL IF-ELSE

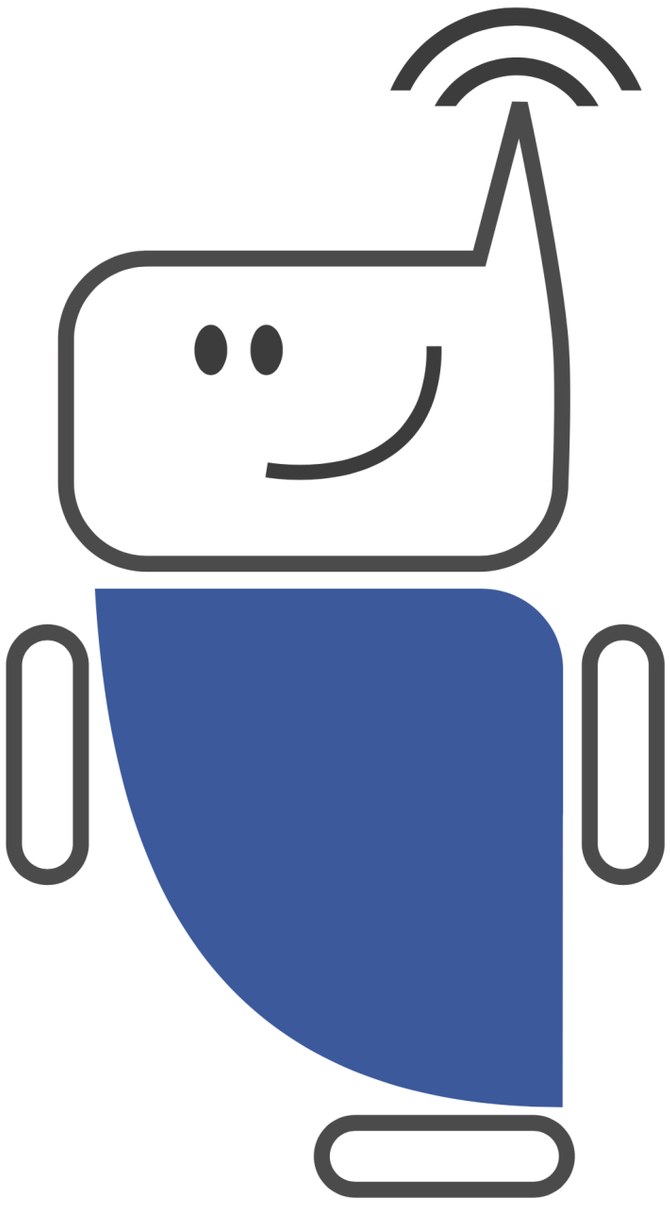


PROF. ELEANDRO MASCHIO



ROTEIRO

- Estrutura Condicional `if-else`
- Omissão do `else`
- Expressões Booleanas
- Aninhamento
- Indentação
- Omissão dos Delimitadores de Bloco
- Um Melhor Uso do `return`
- Princípio da Responsabilidade Única
- Mas... Eu Fiz Assim e Funcionou



ESTRUTURA CONDICIONAL

IF-ELSE

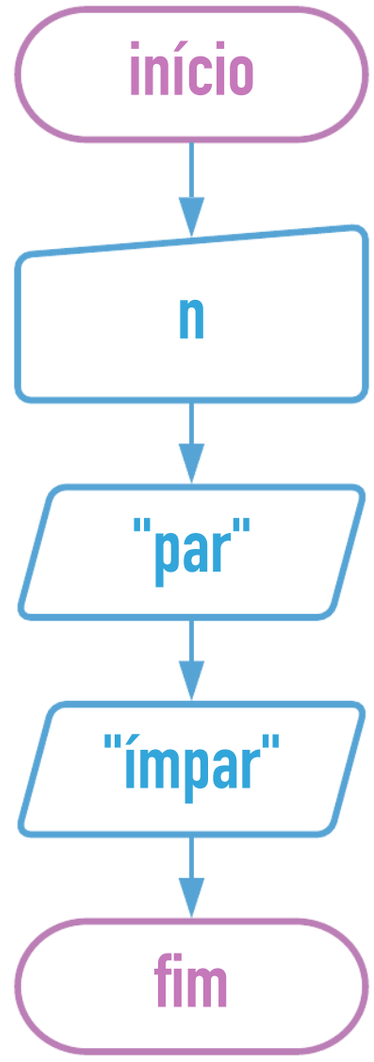
EXEMPLO – PAR OU ÍMPAR

Determine se um número inteiro **n** é par ou ímpar.

INSTRUÇÕES SEQUENCIAIS

Conforme o número inteiro dado, classificá-lo como **par** ou **ímpar**.

- As classificações são **exclusivas**.
  - Retorna-se que o número é ou par, ou ímpar.
- O conjunto de instruções visto até agora **não possibilita** que executemos trechos exclusivos de código.
- Até o presente, temos uma **mera progressão do fluxo de controle**.
  - Ocorre linha após linha.
- Em um mesmo bloco, a linha **sucessora** é executada **incondicionalmente após a linha atual**.



INSTRUÇÕES SEQUENCIAIS

```
// Implementação imperativa

const teclado = require("readline-sync");

let n: number = 0;

console.log("Digite um número inteiro:");
n = teclado.questionInt();

console.log();

console.log("O número digitado é par.");

console.log("O número digitado é ímpar.");
```

INSTRUÇÕES SEQUENCIAIS

```
// Implementação orientada a objetos

export default class NumeroInteiro {
  private n: number;

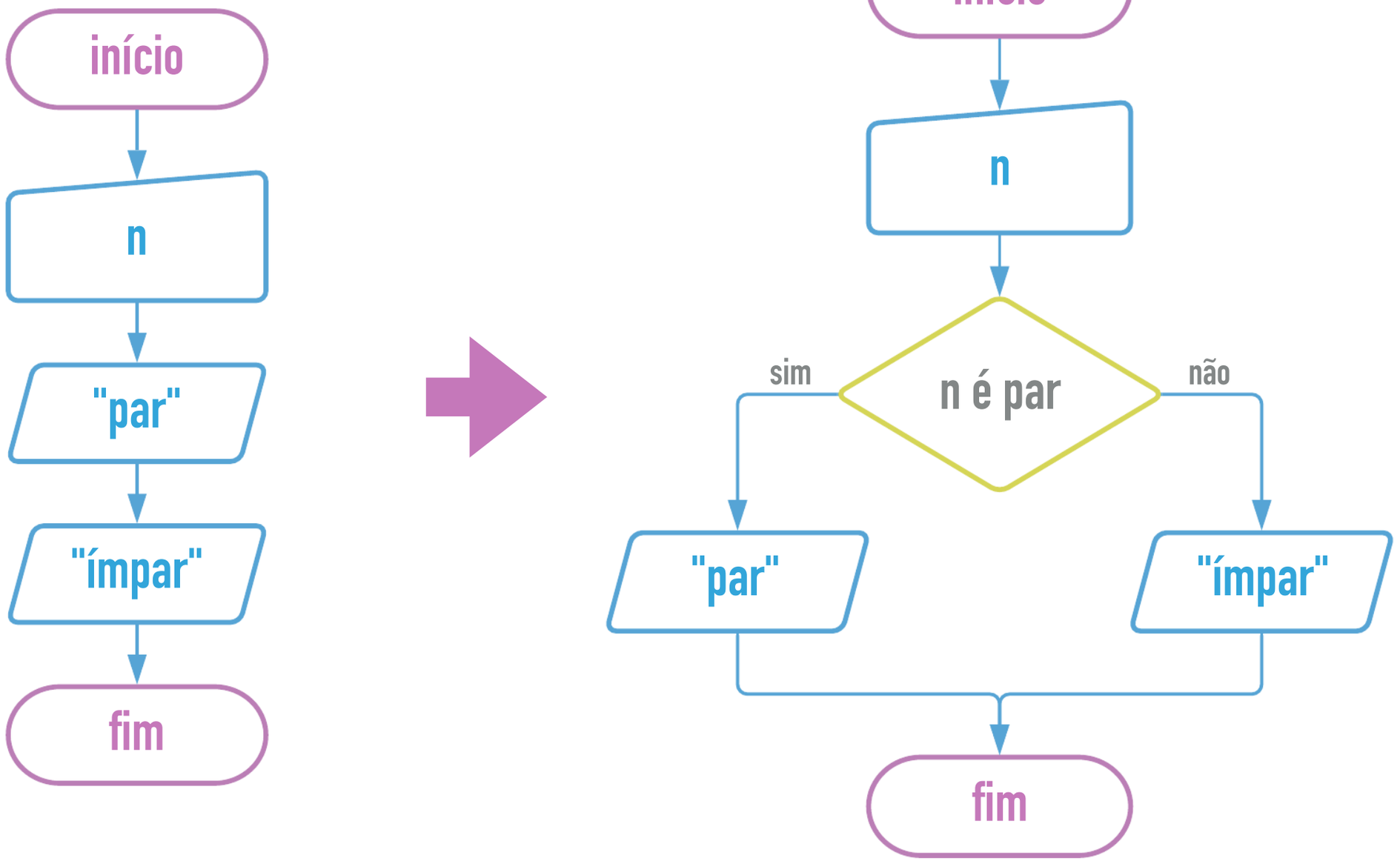
  public constructor(n: number) {
    this.n = n;
  }

  // Implementação de get e set.

  public parOuImpar(): string {
    return "par";

    return "impar";
  }
}
```

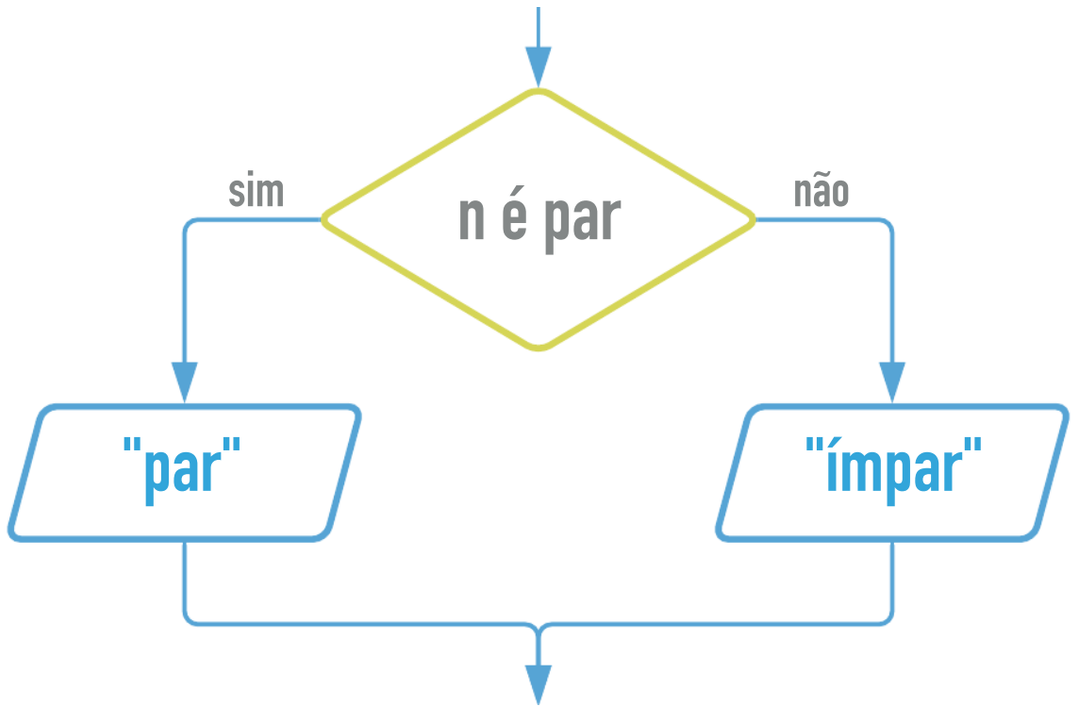
UM DESVIO CONDICIONAL É NECESSÁRIO



UM DESVIO CONDICIONAL É NECESSÁRIO

Deve-se **bifurcar o fluxo de execução** para que, na implementação imperativa:

- **Se** o **número for par**, execute:
  - `console.log("O número digitado é par.");`
- **Caso contrário**, ou seja, se o número for ímpar, execute:
  - `console.log("O número digitado é ímpar.");`

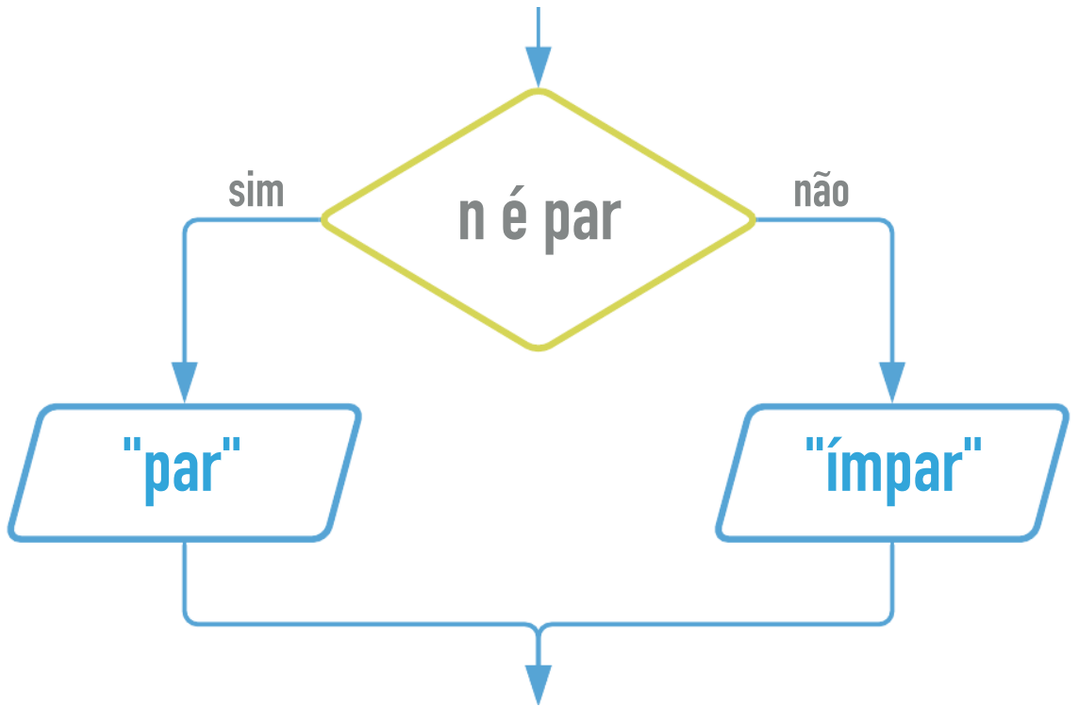


UM DESVIO CONDICIONAL É NECESSÁRIO

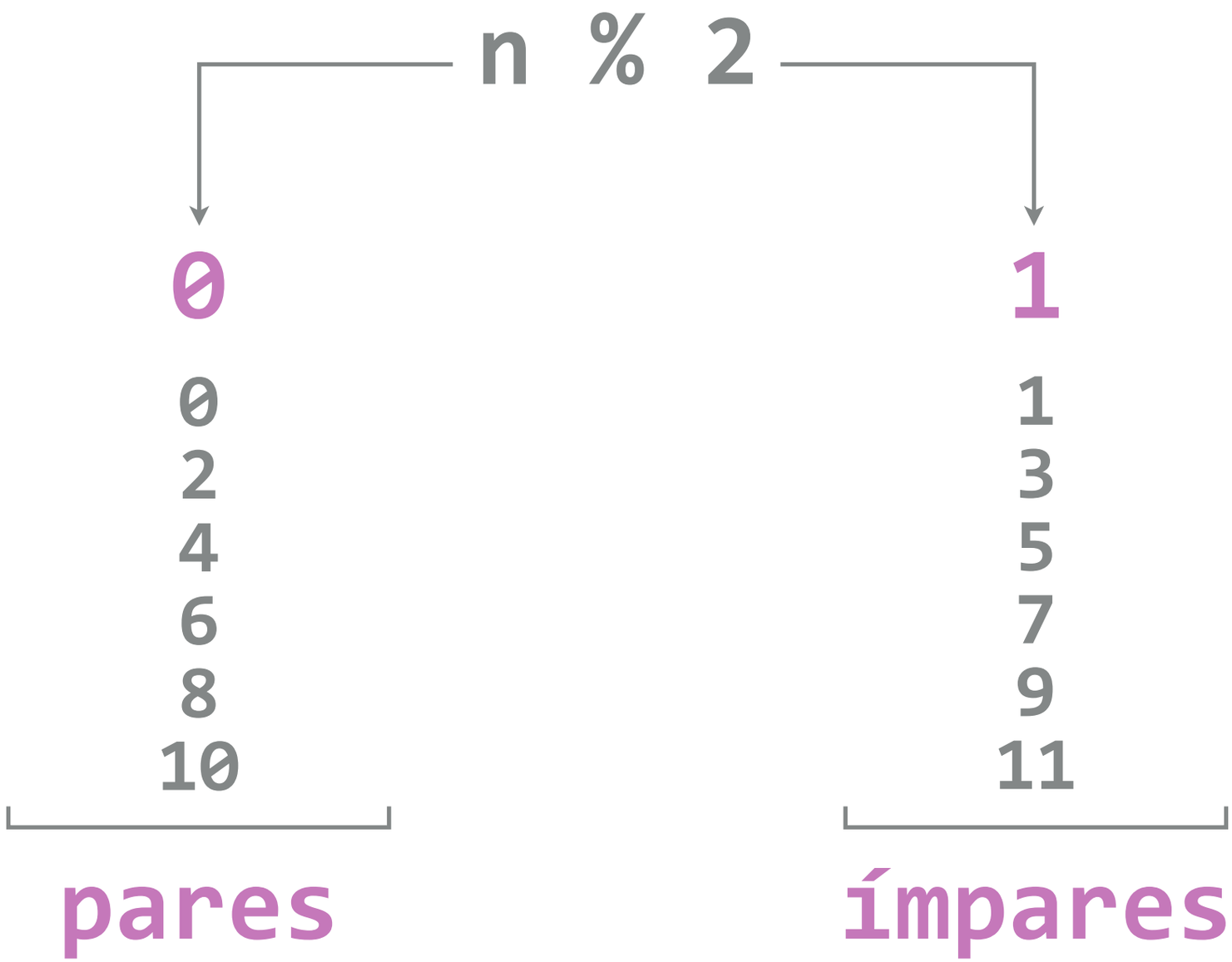
E em uma implementação orientada a objetos:

- **Se** o **número for par**, execute:
  - `return "par";`

- **Caso contrário**, ou seja, se o número for ímpar, execute:
  - `return "ímpar";`



UM DESVIO CONDICIONAL É NECESSÁRIO



UM DESVIO CONDICIONAL É NECESSÁRIO

```
// Implementação imperativa

const teclado = require("readline-sync");

let n: number = 0;

console.log("Digite um número inteiro:");
n = teclado.questionInt();

console.log();

if (n % 2 == 0)
{
  console.log("O número digitado é par.");
}
else
{
  console.log("O número digitado é ímpar.");
}
```

UM DESVIO CONDICIONAL É NECESSÁRIO

```
// Implementação orientada a objetos

export default class NumeroInteiro {
  private n: number;

  public constructor(n: number) {
    this.n = n;
  }

  // Implementação de get e set.

  public parOuImpar(): string {
    if (this.n % 2 == 0)
    {
      return "par";
    }
    else
    {
      return "ímpar";
    }
  }
}
```

LOGO MELHORAREMOS O USO DO RETURN!

ESTRUTURAS CONDICIONAIS

- Possibilitam a utilização de **desvios condicionais (lógicos) progressivos** no código.
- Com isso, pode-se saltar, de maneira progressiva, **blocos de instruções** de acordo com o resultado da condição avaliada.
- **Condição** diz respeito a uma **expressão booleana** que orientará o fluxo de execução para o desvio.



SINTAXE COMPARADA DO IF-ELSE

```
if (condicao)
{
  true
}
else
{
  false
}
```

```
se (condicao)
{
   // bloco de instruções
   // para caso verdadeiro
}
senão
{
   // bloco de instruções
   // para caso falso
}
```

onde **`condicao`** é uma expressão booleana que será avaliada pela estrutura condicional **`if`**.

Como se trata de uma expressão booleana, o resultado da avaliação admite dois valores possíveis: **`true`** ou **`false`**.

SINTAXE EXEMPLIFICADA DO IF-ELSE

```
if (condicao)
{
    // bloco de instruções
    // para caso verdadeiro
}
else
{
    // bloco de instruções
    // para caso falso
}
```

```
if (this.n % 2 == 0)
{
    // caso verdadeiro
    return "par";
}
else
{
    // caso falso
    return "ímpar";
}
```

onde **`condicao`** é uma expressão booleana que será avaliada pela estrutura condicional **`if`**.

Como se trata de uma expressão booleana, o resultado da avaliação admite dois valores possíveis: **`true`** ou **`false`**.

SEMÂNTICA EXEMPLIFICADA DO IF-ELSE



`true`

- O **fluxo de execução** será direcionado ao bloco correspondente ao **caso verdadeiro**.
  - Se o resto da divisão de **n** por 2 for **igual a zero**.

`false`

- Do contrário, o fluxo de execução será direcionado ao bloco do **caso falso**.
  - Se o resto da divisão de **n** por 2 for **diferente de zero**.

SEMÂNTICA EXEMPLIFICADA DO IF-ELSE

```
export default class NumeroInteiro {
  private n: number;

  public constructor(n: number) {
    this.n = n;
  }

  // Implementação de get e set.

  public parOuImpar(): string {
    if (this.n % 2 == 0)
    {
      return "par";
    }
    else
    {
      return "ímpar";
    }
  }
}
```

true

SEMÂNTICA EXEMPLIFICADA DO IF-ELSE

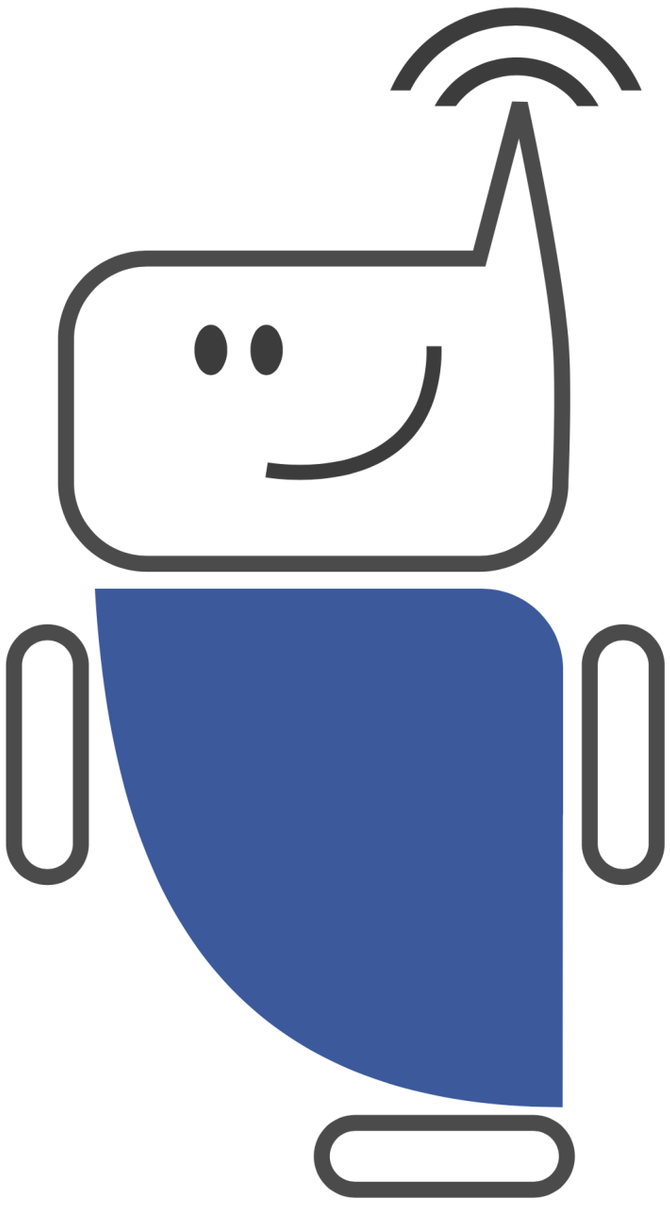
```
export default class NumeroInteiro {
  private n: number;

  public constructor(n: number) {
    this.n = n;
  }

  // Implementação de get e set.

  public parOuImpar(): string {
    if (this.n % 2 == 0)
    {
      return "par";
    }
    else
    {
      return "ímpar";
    }
  }
}
```

false



OMISSÃO DO ELSE

NÃO HÁ INSTRUÇÕES PARA O CASO CONTRÁRIO

- Há **problemas** em que se deseja apenas executar um bloco de instruções caso uma condição seja **verdadeira**.
- Não há bloco de instruções alternativas para o **caso contrário**.
- Pode-se, então, omitir o trecho referente ao `else` (**senão**).

EXEMPLO – VALOR ABSOLUTO

No construtor da classe **`NumeroInteiro`**, faça com que o atributo **`n`** assuma o valor absoluto do parâmetro dado, de maneira semelhante ao método **`Math.abs()`**.

IF COM ELSE OMITIDO

```
export default class NumeroInteiro {
  private n: number;

  public constructor(n: number) {
    if (n < 0)
    {
       n = -n;
    }

    this.n = n;
  }

  // ... outros métodos.
}
```

Idealmente, deve ser abordado pelo Princípio da Responsabilidade Única.

IF COM ELSE OMITIDO

```
export default class NumeroInteiro {
  private n: number;

  public constructor(n: number) {
    if (n < 0)
    {
       n = -n;
    }

    this.n = n;
  }

  // ... outros métodos.
}
```

true

IF COM ELSE OMITIDO

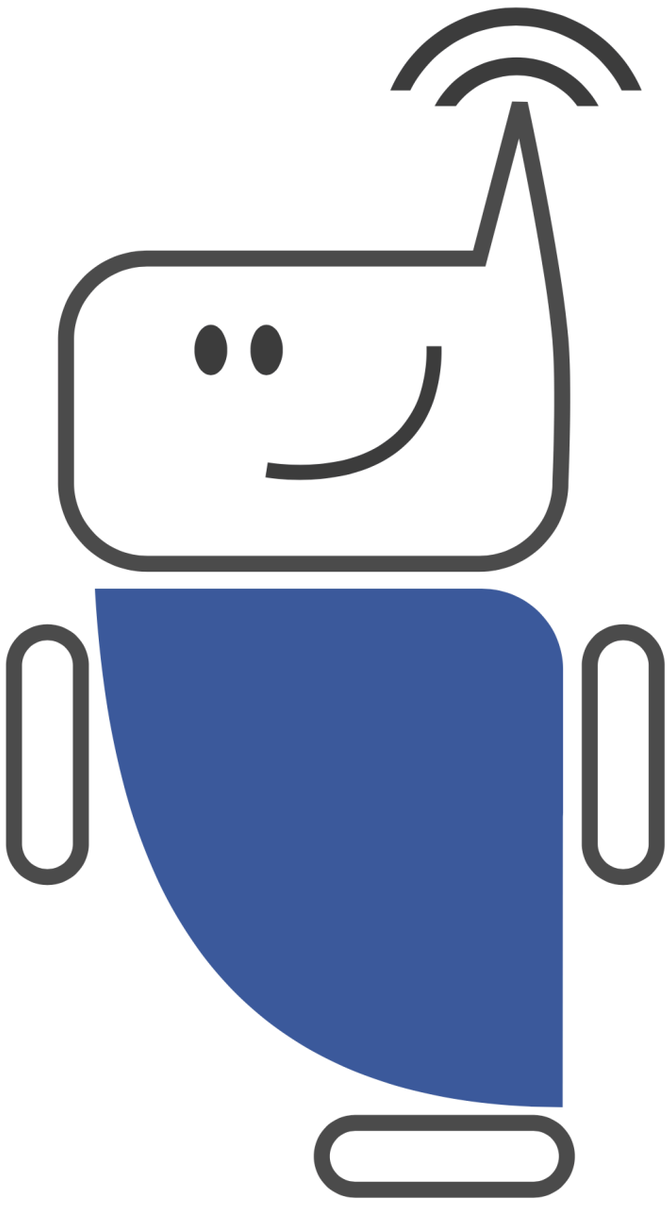
```
export default class NumeroInteiro {
  private n: number;

  public constructor(n: number) {
    if (n < 0)
    {
       n = -n;
    }

    this.n = n;
  }

  // ... outros métodos.
}
```

false

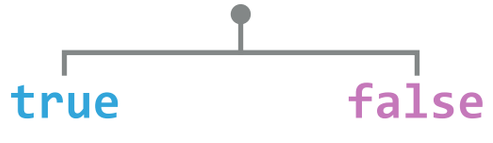


EXPRESSÕES BOOLEANAS

OPERADORES RELACIONAIS

- Comparam a relação entre os operandos.
- São possíveis respostas: `true` e `false`.

OPERADOR	RELAÇÃO	EXEMPLO
==	igual a	`resposta == "s"`
!=	diferente de	`divisor != 2`
>	maior que	`salario > salarioMinimo`
>=	maior ou igual a	`saldo >= valorProduto`
<	menor que	`cotacaoDolar < cotacaoEuro`
<=	menor ou igual a	`percentualFaltas <= 0.25`



OPERADOR RELACIONAL DE IGUALDADE E ATRIBUIÇÃO

ISSO É UMA ATRIBUIÇÃO

```
let x: number = 1;

if (x = 0)
{
  console.log("Entrou!");
}
```

ISSO É UMA COMPARAÇÃO DE IGUALDADE

```
if (x == 0)
{
   // bloco de instruções
}
```

COMPARAÇÃO DE STRINGS

CASO SENSITIVO

```
String senha = "qwerty", confirmacao = new String("Qwerty");

if (senha == confirmacao)
{
    // bloco de instruções
}
```

false

A comparação de **strings** é sensível ou caso (*case sensitive*), ou seja, faz diferenciação entre letras maiúsculas e minúsculas.

USE OS MÉTODOS STRING.TOLOWERCASE( ) E TOUPPERCASE( )

```
if (senha.toLowerCase() == confirmacao.toLowerCase())
{
    // bloco de instruções
}
```

true

O método **toLowerCase()** converte a **string** para minúsculas.

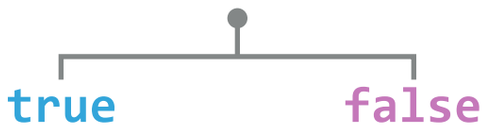
OPERADORES BOOLEANOS

- A estrutura condicional **`if-else`** exige uma expressão booleana (condição) para determinar quais instruções devem ser executadas.
- As **condições simples** conseguem ser expressas em termos dos **operadores relacionais**.
- Mas cada expressão formada por um operador relacional **testa apenas uma condição**.
- O teste de **múltiplas condições**, muitas vezes, requer que uma expressão mais complexa seja construída.
- Operadores booleanos (ou lógicos) permitem que atuemos sobre valores (e expressões que retornem) **`true`** ou **`false`**.
  - Da mesma forma que os **operadores aritméticos** permitem que manipulemos valores e expressões numéricas.

OPERADORES BOOLEANOS

Principais operadores booleanos do TypeScript para a estrutura condicional **`if-else`**:

OPERADOR	SIGNIFICADO	PORTUGUÊS	INGLÊS
`!`	negação ou complemento lógico	`não`	`not`
`&&`	conjunção lógica	`e`	`and`
`||`	disjunção lógica	`ou`	`or`



OPERADOR BOOLEANO DE NEGAÇÃO !

- Negação ou complemento lógico: **não**, ***not*** (unário).
- **Inverte** o resultado de uma expressão booleana.
- Sendo **a** uma expressão booleana, a expressão **(!a)** é:
  - **verdade** se **a** for falsa.
  - **falsa** se **a** for verdadeira.

a	!a
`true`	`false`
`false`	`true`

TABELA VERDADE

OPERADOR BOOLEANO DE NEGAÇÃO !

EXEMPLO

Vou ao mercado se eu não estiver com fome.

- Se eu estiver com fome, não vou ao mercado.
- Se eu não estiver com fome, eu vou ao mercado.

```
mercado = !fome
```

fome	mercado
true	false
false	true

TABELA VERDADE

OPERADOR BOOLEANO DE CONJUNÇÃO &&

- Conjunção lógica: **e**, ***and*** (binário).
- Sendo **a** e **b** duas expressões booleanas, **(a && b)** é:
  - **verdade** apenas quanto tanto **a** quanto **b** forem verdadeiras.
  - **falsa** para quaisquer outros valores de **a** e **b**.

a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

TABELA VERDADE

OPERADOR BOOLEANO DE CONJUNÇÃO &&

EXEMPLO

Viajamos para a praia se fizer sol e conseguirmos carona.

- Se houver sol e carona, vamos para a praia.
- Ao contrário, não vamos.

```
praia = sol && carona
```

(mais restritivo)

sol	carona	praia
true	true	true
true	false	false
false	true	false
false	false	false

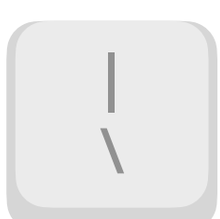
TABELA VERDADE

OPERADOR BOOLEANO DE DISJUNÇÃO ||

- Disjunção lógica: **ou**, ***or*** (binário).
- A expressão (`a || b`) é:
  - **verdade** quando ao menos uma uma das expressões, **a** ou **b**, for verdadeira.
  - **falsa** somente quando tanto **a** quanto **b** forem falsas.

a	b	a || b
`true`	`true`	`true`
`true`	`false`	`true`
`false`	`true`	`true`
`false`	`false`	`false`

TABELA VERDADE



OPERADOR BOOLEANO DE DISJUNÇÃO ||

EXEMPLO

Viajamos para a praia se fizer sol e conseguirmos carona.

- Se houver sol, ou conseguirmos carona, vamos para a praia.
- Se não houver nenhum, não vamos.

```
praia = sol || carona
```

(mais permissivo)

sol	carona	praia
true	true	true
true	false	true
false	true	true
false	false	false

TABELA VERDADE

COMPARATIVO ENTRE OS OPERADORES BOOLEANOS

- **Negação**
  Inverte o operando.
- **Conjunção**
  Verdadeira quando ambos os operandos forem verdadeiros.
- **Disjunção**
  Falsa quando ambos os operandos forem falsos.

a	!a
true	false
false	true

TABELA VERDADE

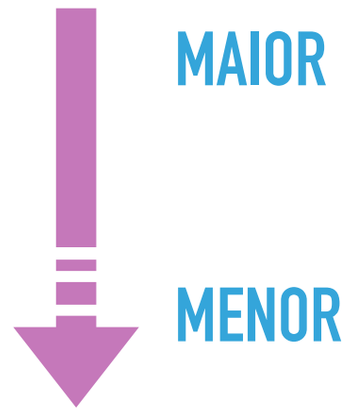
a	b	a && b	a || b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

TABELA VERDADE

PRECEDÊNCIA DOS OPERADORES BOOLEANOS

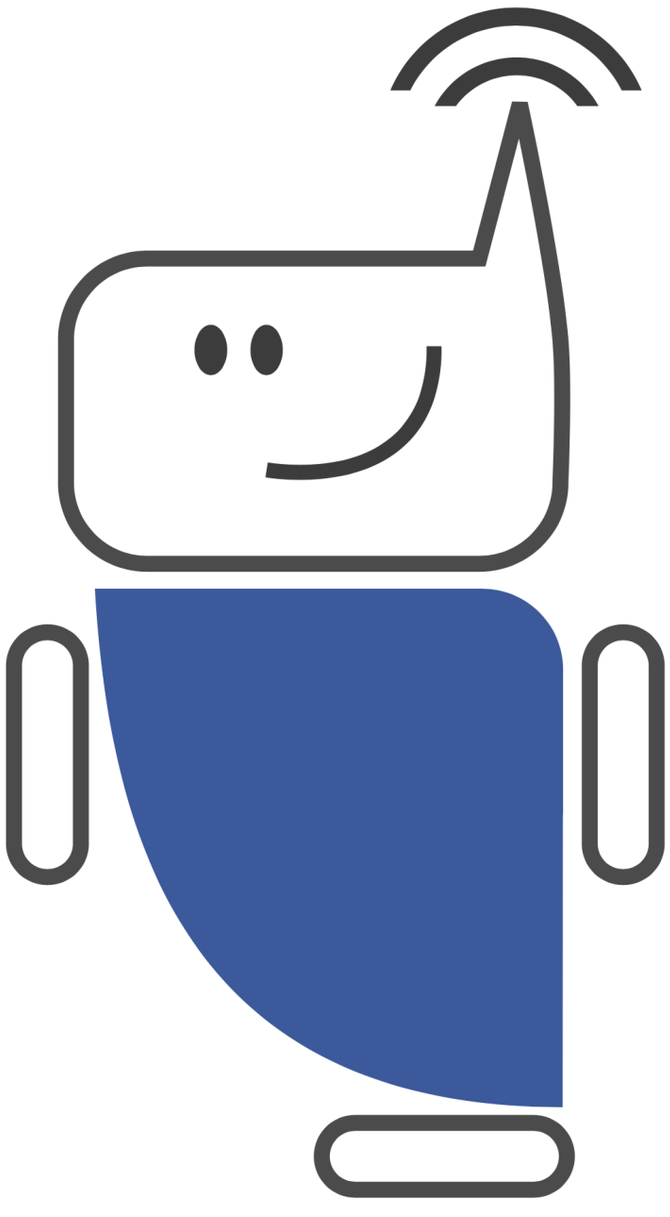
- Operadores de mesma precedência são avaliados na ordem em que se fizerem presentes, da esquerda para a direita.

PRECEDÊNCIA	OPERADOR
1°	!
2°	&&
3°	||



EXEMPLOS DE EXPRESSÕES BOOLEANAS

- `!(numerador == denominador) // equivale a (numerador != denominador)`
- `!(senha.length == 0)`
- `(senha == confirmacao) && !(senha.length == 0)`
- `nota >= 6 && percentualFaltas <= 0.25`
- `cotacaoDolar < cotacaoEuro || cotacaoLibra < cotacaoEuro`
- `(qtde < qtdeMaxima) && (saldo > 0 || credito > 0)`



ANINHAMENTO

EXEMPLO – POSITIVO, NEGATIVO OU NULO

Determine se um número inteiro **n** é estritamente positivo, negativo ou nulo.

APENAS UMA ESTRUTURA IF-ELSE É INSUFICIENTE

Conforme o número inteiro dado, classificá-lo como **positivo**, **negativo** ou **nulo**.

- Um único `if-else` permite somente as alternativas de **caso verdadeiro** e **caso falso** para uma condição.

```
if (condicao)
{
    true
}
else
{
    false
}
```

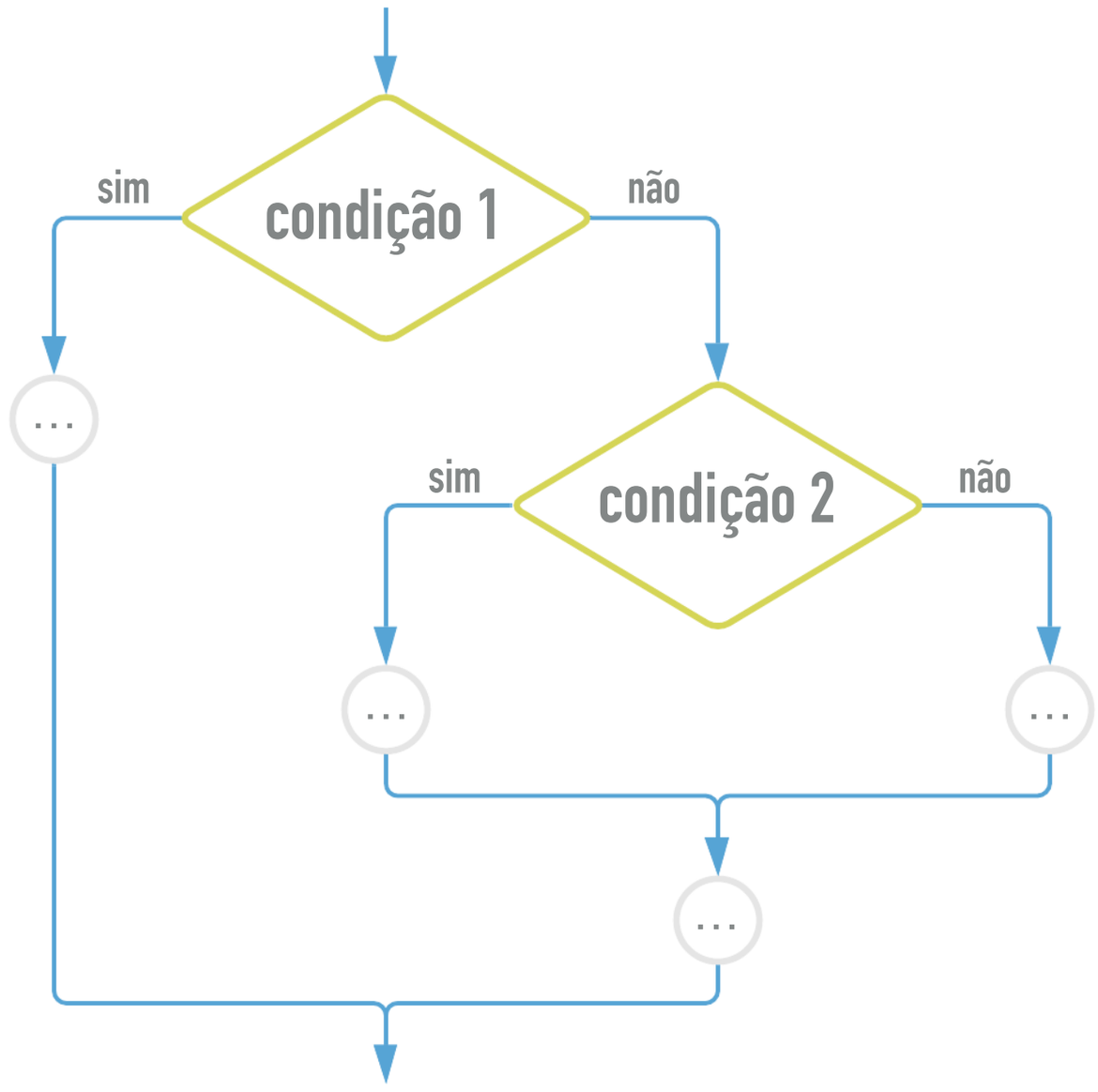


DOIS DESVIOS CONDICIONAIS SÃO NECESSÁRIOS

```
if (condicao1)
{
   true
}
else
{
   false

   if (condicao2)
   {
      true
   }
   else
   {
      false
   }
}
```





DOIS DESVIOS CONDICIONAIS SÃO NECESSÁRIOS

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

LOGO MELHORAREMOS O USO DO RETURN!

CONFORME NECESSÁRIO, É POSSÍVEL COMBINAR MAIS DE UM DESVIO CONDICIONAL

TESTE DE MESA

sendo this.n = 0

```
    public positivoNegativoOuNulo(): string {
      if (this.n > 0)
      {
        return "positivo";
      }
      else
      {
        if (this.n < 0)
        {
          return "negativo";
        }
        else
        {
          return "nulo";
        }
      }
}
```

n	n > 0	n < 0	return
0			

TESTE DE MESA

TESTE DE MESA

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

n	n > 0	n < 0	return
0	false		

TESTE DE MESA

TESTE DE MESA

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

n	n > 0	n < 0	return
0	false		

TESTE DE MESA

TESTE DE MESA

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

n	n > 0	n < 0	return
0	false	false	

TESTE DE MESA

TESTE DE MESA

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

n	n > 0	n < 0	return
0	false	false	

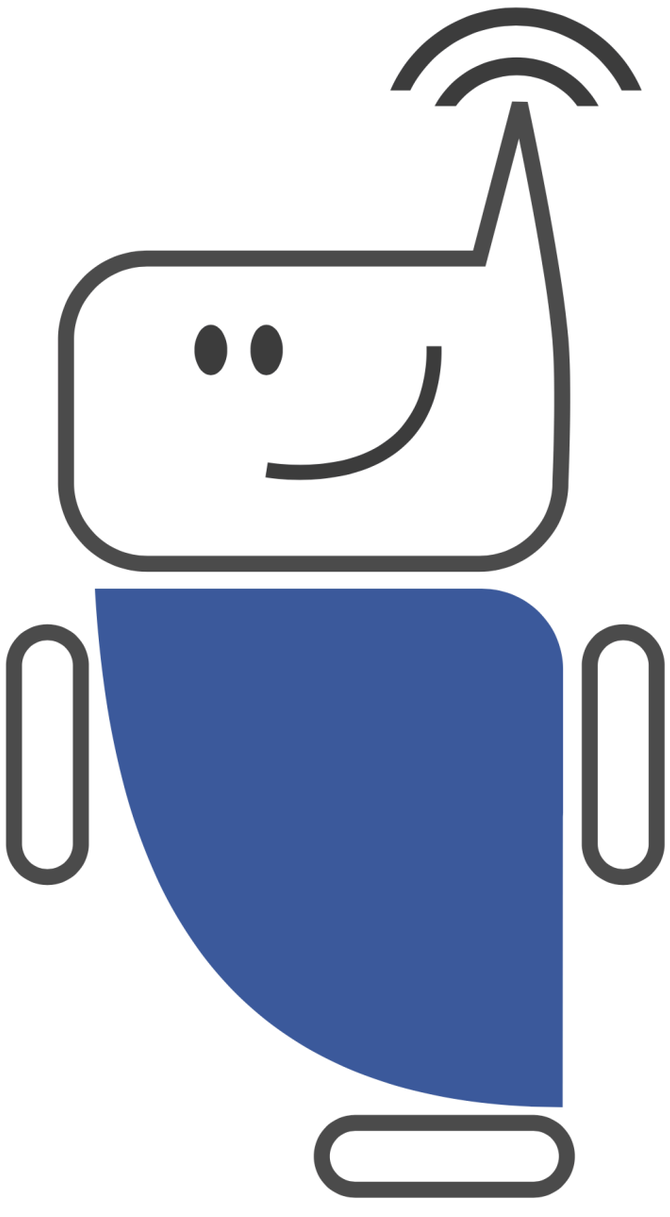
TESTE DE MESA

TESTE DE MESA

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

n	n > 0	n < 0	return
0	false	false	"nulo"

TESTE DE MESA



INDENTAÇÃO

É FÁCIL DE ENTENDER, OU DE ALTERAR, O TRECHO DE CÓDIGO SEGUINTE?

SEM INDENTAÇÃO

```
public positivoNegativoOuNulo(): string {
if (this.n > 0)
{
return "positivo";
}
else
{
if (this.n < 0)
{
return "negativo";
}
else
{
return "nulo";
}
}
}
```

E AGORA, COM INDENTAÇÃO?

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

O PORQUÊ DA INDENTAÇÃO

- Favorece a **legibilidade**.
- Destaca a estrutura hierárquica do código por meio de **recuos**.
- São incrementados após o início de cada estrutura.
- Depois, decrementados antes do fim de cada estrutura.
- Usam-se entre **dois** e **quatro** espaços.
- Deve-se:
  - **Uniformizar** a quantidade de espaços ao longo do código.
  - Observar as **convenções** adotadas onde estiver programando.

FACILITADORES PARA A INDENTAÇÃO

- Em TypeScript, prefere-se não usar a tecla **Tab** ou (⇆).
- Editores têm a opção de substituir automaticamente tabulações por espaços:
  - Parâmetro **`tabsize = 4`**.
  - Significa que a tecla **tab** será substituída por **4 espaços**.
- Indentação automática:
  - Eclipse: Ctrl + Shift + F.
  - NetBeans: Alt + Shift + F.
  - Sublime: Edit \ Line \ Reindent.
  - Visual Studio Code: Ctrl + S (automático ao salvar).
    - Instalar a extensão **Prettier - Code formatter**.

HÁ DOIS TIPOS DE PESSOAS

```
public positivoNegativoOuNulo(): string
{
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

- Favorece a visualização hierárquica.
- Facilita visualizar onde as chaves foram abertas.
- Codificação mais extensa.

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0) {
    return "positivo";
  } else if (this.n < 0) {
    return "negativo";
  } else {
    return "nulo";
  }
}
```

- Padrão utilizado na web (formatação do Prettier).
- Dificulta visualizar onde as chaves foram abertas.
- Codificação mais abreviada.

NAS ESTRUTURAS CONDICIONAIS DOS SLIDES FOI USADA A FORMATAÇÃO DE CHAVE-EMBAIXO-DE-CHAVE, PARA DESTACAR A HIERARQUIA.

TABULAÇÃO (TAB)

Linguagem Python.

- A indentação **não é elemento estético** que favorece a legibilidade.
- Mas um elemento **sintático da linguagem**.
- A hierarquia das instruções é expressada pela **tabulação**.
- Não usar tabulação implica:
  - Erros; ou
  - Comportamentos indesejáveis.



EXEMPLO – QUESTÃO TÍPICA DE CONCURSO

Considere o seguinte trecho de código, implementado em TypeScritpt. Qual é o valor de **aux** ao final da execução?

```
let a = 8, b = 16, c = 4, aux = 0;
if (a > b)
{
aux = a;
a = b;
b = aux;
}
if (b > c)
{
aux = b;
b = c;
c = aux;
if (a > b)
{
aux = a;
a = b;
b = aux;
}
}
```

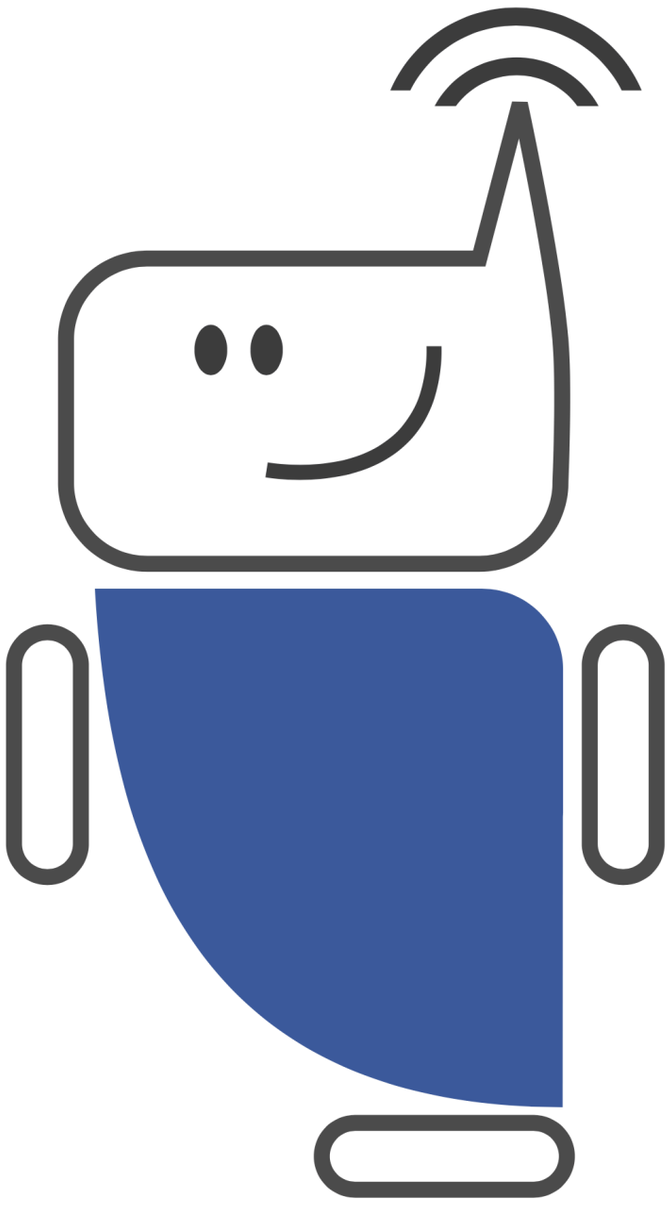
(a) 0

(b) 4

(c) 8

(d) 16

(e) ocorre erro de compilação



OMISSÃO DOS

DELIMITADORES DE BLOCO

QUANDO É POSSÍVEL OMITIR?

- Os delimitadores de bloco, { e }, podem ser omitidos em uma estrutura condicional quando houver **somente uma instrução** no bloco.

- No início, como são opcionais, é uma **medida de segurança** utilizá-los mesmo assim.
  - Permanecem já como garantia para que outras instruções sejam inseridas no futuro.

- Com um pouco de prática, a omissão torna-se natural.

- Recurso de autocompletar das IDEs.
  - `if` + **Tab** (⇤⇥).

EXEMPLO

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

EXEMPLO

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
  {
    return "positivo";
  }
  else
  {
    if (this.n < 0)
    {
      return "negativo";
    }
    else
    {
      return "nulo";
    }
  }
}
```

EXEMPLO

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";
  else
  {
    if (this.n < 0)
      return "negativo";
    else
      return "nulo";
  }
}
```

EXEMPLO

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";
  else
  {
    if (this.n < 0)
      return "negativo";
    else
      return "nulo";
  }
}
```

EXEMPLO

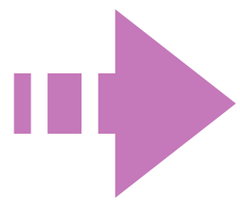
```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";
  else
    if (this.n < 0)
      return "negativo";
    else
      return "nulo";
}
```

CUIDADO COM AS ALTERAÇÕES

```
if (condicao)
{
   instrucao1;
}
```

CUIDADO COM AS ALTERAÇÕES

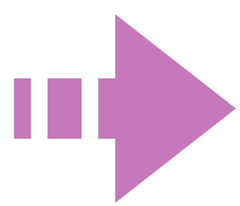
```
if (condicao)
{
    instrucao1;
}
```



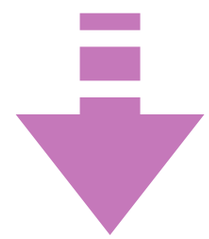
```
if (condicao)
    instrucao1;
```

CUIDADO COM AS ALTERAÇÕES

```
if (condicao)
{
    instrucao1;
}
```



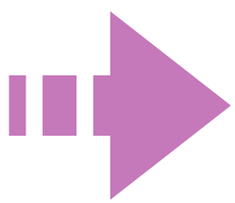
```
if (condicao)
└── instrucao1;
```



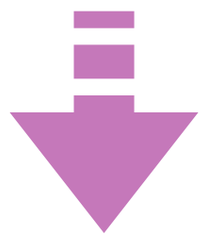
```
if (condicao)
└── instrucao1;
    instrucao2;
    instrucao3;
    instrucao4;
```

CUIDADO COM AS ALTERAÇÕES

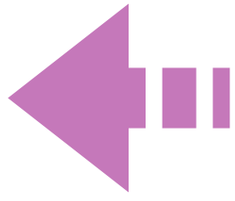
```
if (condicao)
{
    instrucao1;
}
```



```
if (condicao)
    instrucao1;
```



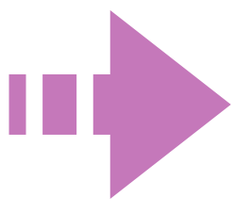
```
if (condicao)
    instrucao1;
    instrucao2;
    instrucao3;
    instrucao4;
```



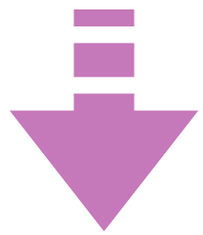
```
if (condicao)
    instrucao1;
instrucao2;
instrucao3;
instrucao4;
```

CUIDADO COM AS ALTERAÇÕES

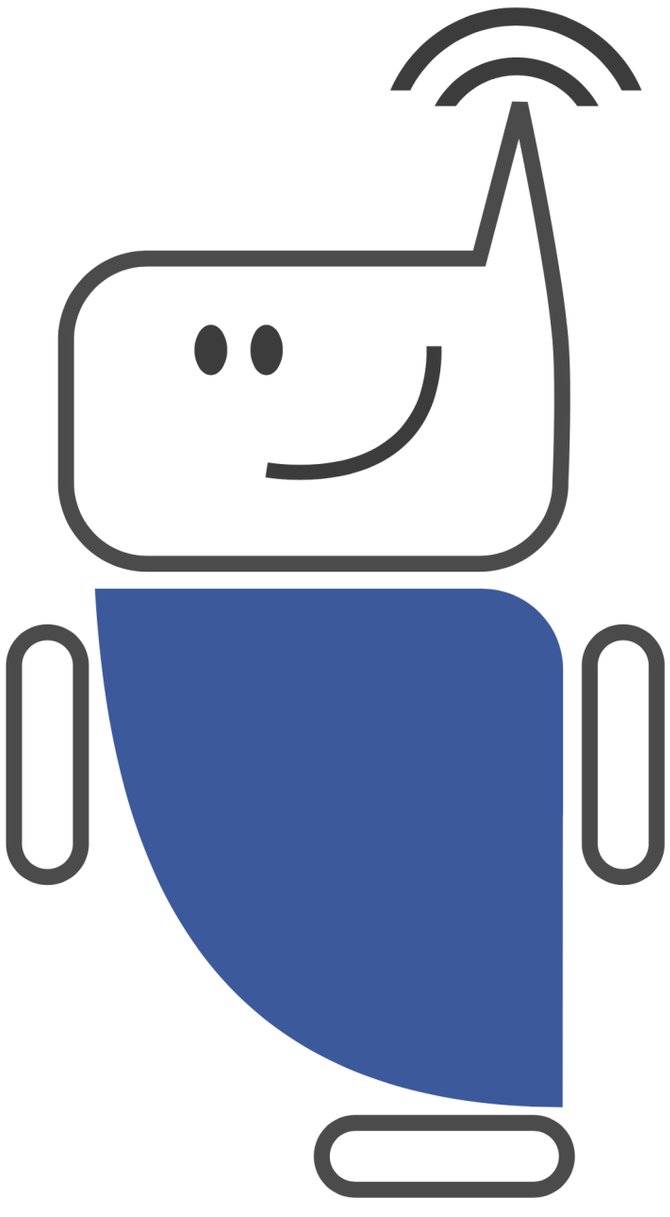
```
if (condicao)
{
    instrucao1;
}
```



```
if (condicao)
    instrucao1;
```



```
if (condicao)
{
    instrucao1;
    instrucao2;
    instrucao3;
    instrucao4;
}
```



UM MELHOR USO DO

RETURN

INSTRUÇÃO RETURN

- Um **método retorna** ao código que o invocou quando:

    1. termina de executar as instruções contidas;
    2. encontra um **`return`**; ou
    3. lança uma exceção (abordado futuramente por IOO).

- Qualquer um desses que ocorra primeiro.

- Existem poréns...

INSTRUÇÃO RETURN

- O **tipo de retorno** é declarado na definição de um método.
- Um método declarado como **`void`** não retorna qualquer valor.
  - Portanto, não precisa conter uma instrução **`return`**.
- Mas pode conter.
  - O **`return`** é usado para abandonar a execução daquele método naquele exato ponto do código.
- Use apenas:

  `return;`
- A tentativa de retornar um valor em um método **`void`** ocasiona um erro de compilação.

INSTRUÇÃO RETURN

- Em qualquer outro método não declarado como **`void`**, **deve haver uma instrução de retorno** com o valor correspondente:

```
return valor;
```

- O **tipo de dado** do valor de retorno precisa ser compatível com o tipo de retorno declarado no método.
  - Não se pode retornar um **`number`** em um método declarado para retornar um **`boolean`**.

ABANDONO DA EXECUÇÃO

O ABANDONO DA EXECUÇÃO POR MEIO DO RETURN É UM RECURSO QUE PODE MELHORAR MUITO A QUALIDADE E A LEGIBILIDADE DO CÓDIGO

EXEMPLO – APRIMORAR RESOLUÇÃO COM RETURN

Aprimore o método **`positivoNegativoOuNulo`**`()` com o uso correto do **`return`**.

EXEMPLO

- Se o fluxo de execução chegou até a instrução destacada, **`n`** é maior do que zero.
- Com isso, a *string* **`"positivo"`** é retornada à instrução que invocou o método.
- A execução do método **`positivoNegativoOuNulo()`** será abandonada.

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";
  else
  {
    if (this.n < 0)
      return "negativo";
    else
      return "nulo";
  }
}
```

EXEMPLO

- Se o fluxo de execução foi além e chegou até o **`else`**, significa **`n`** é menor ou igual zero.
- Entretanto, o mais importante é observar que, se o fluxo de execução chegou nesta linha, a **execução do método não foi abandonada** (e o número não era positivo).
- Assim, o **`else`** é desnecessário (pois se a execução do método continuou, **`n`** não é positivo).

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";
  else
  {
    if (this.n < 0)
      return "negativo";
    else
      return "nulo";
  }
}
```

EXEMPLO

- O primeiro **`else`** foi omitido.
- Se **`n`** fosse maior do que zero, teria retornado **`"positivo"`**.
- Caso aconteça o contrário, o fluxo de execução segue para a instrução seguinte.

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";

  if (this.n < 0)
    return "negativo";
  else
    return "nulo";
}
```

EXEMPLO

- Se o fluxo de execução chegou até a instrução destacada, **`n`** é menor do que zero.
- Com isso, a *string* **`"negativo"`** é retornada à instrução que invocou o método.
- A execução do método **`positivoNegativoOuNulo()`** será abandonada.

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";

  if (this.n < 0)
    return "negativo";
  else
    return "nulo";
}
```

EXEMPLO

- Se o fluxo de execução chegou até o segundo **`else`**, significa **n** é igual zero.
- E, o mais importante, se o fluxo de execução chegou nesta linha, a **execução do método não foi abandonada** (e o número não era positivo, nem negativo).
- Assim, esse **`else`** também é desnecessário (pois se a execução do método continuou, **n** é nulo).

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";

  if (this.n < 0)
    return "negativo";
  else
    return "nulo";
}
```

EXEMPLO

- O segundo **`else`** foi omitido.
- Se o fluxo de execução chegou até a instrução destacada, **`n`** é nulo.
- A *string* **`"nulo"`** é retornada à instrução que invocou o método.
- A execução do método **`positivoNegativoOuNulo()`** será abandonada.

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";

  if (this.n < 0)
    return "negativo";
  else
    return "nulo";
}
```

EXEMPLO

- Cada instrução de **`return`**, se alcançada pelo fluxo de execução, **abandona o método** naquele instante.
- Se isso ocorrer, qualquer instrução seguinte será **ignorada**.
- E caso não ocorra, sabemos que qualquer **`return`** anterior **não foi executado**.

```
public positivoNegativoOuNulo(): string {
  if (this.n > 0)
    return "positivo";

  if (this.n < 0)
    return "negativo";

  return "nulo";
}
```

EVOLUÇÃO DO CÓDIGO

```
public positivoNegativoOuNulo(): string
{

   if (this.n > 0)
   {
      return "positivo";
   }
   else
   {
      if (this.n < 0)
      {
         return "negativo";
      }
      else
      {
         return "nulo";
      }
   }

}
```



```
{

   if (this.n > 0)
     return "positivo";
   else
      if (this.n < 0)
        return "negativo";
      else
        return "nulo";

}
```



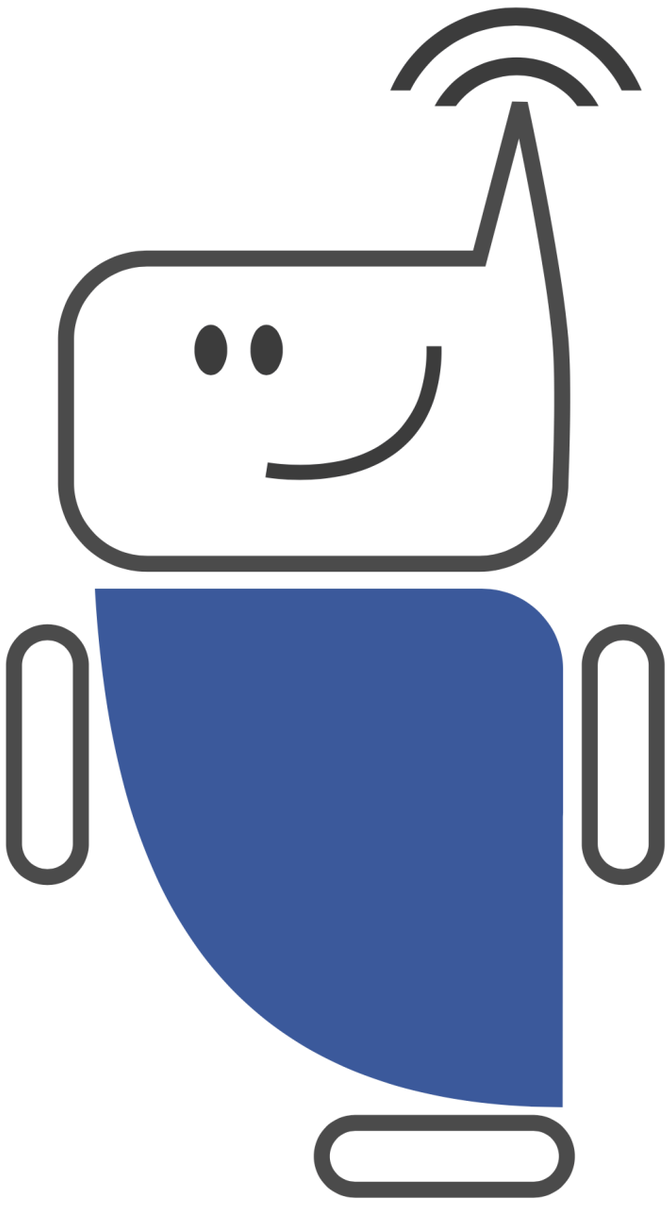
```
{

   if (this.n > 0)
     return "positivo";

   if (this.n < 0)
     return "negativo";

   return "nulo";

}
```



PRINCÍPIO DA

RESPONSABILIDADE ÚNICA

EXEMPLO – REAJUSTE DE SALÁRIO

Considere uma classe que modele um funcionário.
Implemente um método que retorne o salário reajustado, conforme a tabela abaixo:

- até $1.000 (limite aberto), reajuste de 200%;
- de $1.000 até $2.000, reajuste de 100%;
- de $2.000 até $3.000, reajuste de 50%;
- de $3.000 até $4.000, reajuste de 20%;
- acima de $4.000, reajuste de 10%.

IMPLEMENTAÇÃO DA CLASSE FUNCIONARIO

```
export default class Funcionario {
  private salario: number;

  public constructor(salario: number) {
    this.salario = salario;
  }

  public getSalario(): number {
    return this.salario;
  }

  public setSalario(salario: number): void {
    this.salario = salario;
  }


  // Novo(s) método(s)
}
```

TENTATIVA 1

```
public getNovoSalario(): number {
  if (this.salario < 1000)
    return this.salario + (this.salario * 2.0);

  if (this.salario < 2000)
    return this.salario + this.salario;

  if (this.salario < 3000)
    return this.salario + (this.salario * 0.5);

  if (this.salario < 4000)
    return this.salario + (this.salario * 0.2);

  return this.salario + (this.salario * 0.1);
}
```

Fazer uso correto do **return**.

ESTRATÉGIA

TENTATIVA 1

```
public getNovoSalario(): number {
  if (this.salario < 1000)
    return this.salario + (this.salario * 2.0);

  if (this.salario < 2000)
    return this.salario + this.salario;

  if (this.salario < 3000)
    return this.salario + (this.salario * 0.5);

  if (this.salario < 4000)
    return this.salario + (this.salario * 0.2);

  return this.salario + (this.salario * 0.1);
}
```

- O cálculo é repetido 5 vezes.
- Dificuldade de manter a consistência no caso de alterações, pois são 5 lugares para alterar.
- O segundo cálculo não é exatamente igual.

PROBLEMAS

TENTATIVA 2

```
public getNovoSalario(): number {
  if (this.salario < 1000)
    return this.salario * 3.0;

  if (this.salario < 2000)
    return this.salario * 2.0;

  if (this.salario < 3000)
    return this.salario * 1.5;

  if (this.salario < 4000)
    return this.salario * 1.2;

  return this.salario * 1.1;
}
```

Simplificar e uniformizar o cálculo.

ESTRATÉGIA

TENTATIVA 2

```
public getNovoSalario(): number {
  if (this.salario < 1000)
    return this.salario * 3.0;

  if (this.salario < 2000)
    return this.salario * 2.0;

  if (this.salario < 3000)
    return this.salario * 1.5;

  if (this.salario < 4000)
    return this.salario * 1.2;

  return this.salario * 1.1;
}
```

- O cálculo ainda está repetido 5 vezes, mas foi abreviado.
- Continuam sendo 5 lugares para alterar.
- Todos os cálculos estão iguais, mas foi perdida a legibilidade do percentual de reajuste.

PROBLEMAS

TENTATIVA 3

```
public getNovoSalario(): number {
  let reajuste: number = 0;

  if (this.salario < 1000)
    reajuste = 3.0;
  else
    if (this.salario < 2000)
      reajuste = 2.0;
    else
      if (this.salario < 3000)
        reajuste = 1.5;
      else
        if (this.salario < 4000)
          reajuste = 1.2;
        else
          reajuste = 1.1;

  return this.salario * reajuste;
}
```

Fazer o cálculo uma única vez.

ESTRATÉGIA

TENTATIVA 3

- O código avança por 5 níveis hierárquicos (horizontalização).
- A legibilidade do código foi ainda mais prejudicada.

PROBLEMAS



```
public getNovoSalario(): number {
  let reajuste: number = 0;

  if (this.salario < 1000)
    reajuste = 3.0;
  else
    if (this.salario < 2000)
      reajuste = 2.0;
    else
      if (this.salario < 3000)
        reajuste = 1.5;
      else
        if (this.salario < 4000)
          reajuste = 1.2;
        else
          reajuste = 1.1;

  return this.salario * reajuste;
}
```

TENTATIVA 4

```
private getPercentual(): number {
  if (this.salario < 1000)
    return 2.0;

  if (this.salario < 2000)
    return 1.0;

  if (this.salario < 3000)
    return 0.5;

  if (this.salario < 4000)
    return 0.2;

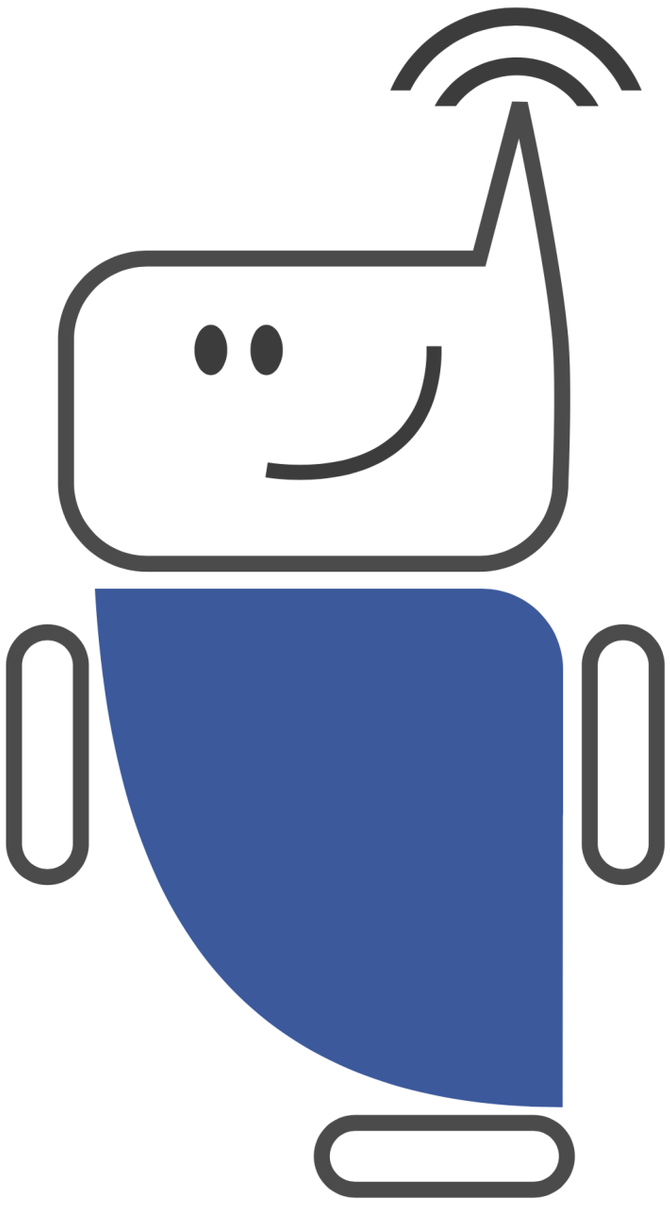
  return 0.1;
}

public getNovoSalario(): number {
  return this.salario * (1 + this.getPercentual());
}
```

Usar o Princípio da Responsabilidade Única:

- Um método para **retornar o percentual**.
- Outro método para **calcular o novo salário**.

ESTRATÉGIA



MAS...

EU FIZ ASSIM E FUNCIONOU

SIM, "FUNCIONA"

```
public getNovoSalario(): number {
  let novoSalario: number = 0;

  if (this.salario < 1000)
  {
    novoSalario = this.salario + (this.salario * 2.0);
  }

  if (this.salario >= 1000 && this.salario < 2000)
  {
    novoSalario = this.salario + this.salario;
  }

  if (this.salario >= 2000 && this.salario < 3000)
  {
    novoSalario = this.salario + (this.salario * 0.5);
  }

  if (this.salario >= 3000 && this.salario < 4000)
  {
    novoSalario = this.salario + (this.salario * 0.2);
  }

  if (this.salario >= 4000)
  {
    novoSalario = this.salario + (this.salario * 0.1);
  }

  return novoSalario;
}
```

SIMULE O FLUXO DE EXECUÇÃO CASO O FUNCIONÁRIO RECEBA MENOS DE $1.000

CÓDIGO MUITO SUSCETÍVEL A ERROS

```
public getNovoSalario(): number {
  let novoSalario: number = 0;

  if (this.salario < 1000)
  {
    novoSalario = this.salario + (this.salario * 2.0);
  }

  if (this.salario >= 1000 && this.salario < 2000)
  {
    novoSalario = this.salario + this.salario;
  }

  if (this.salario >= 2000 && this.salario < 3000)
  {
    novoSalario = this.salario + (this.salario * 0.5);
  }

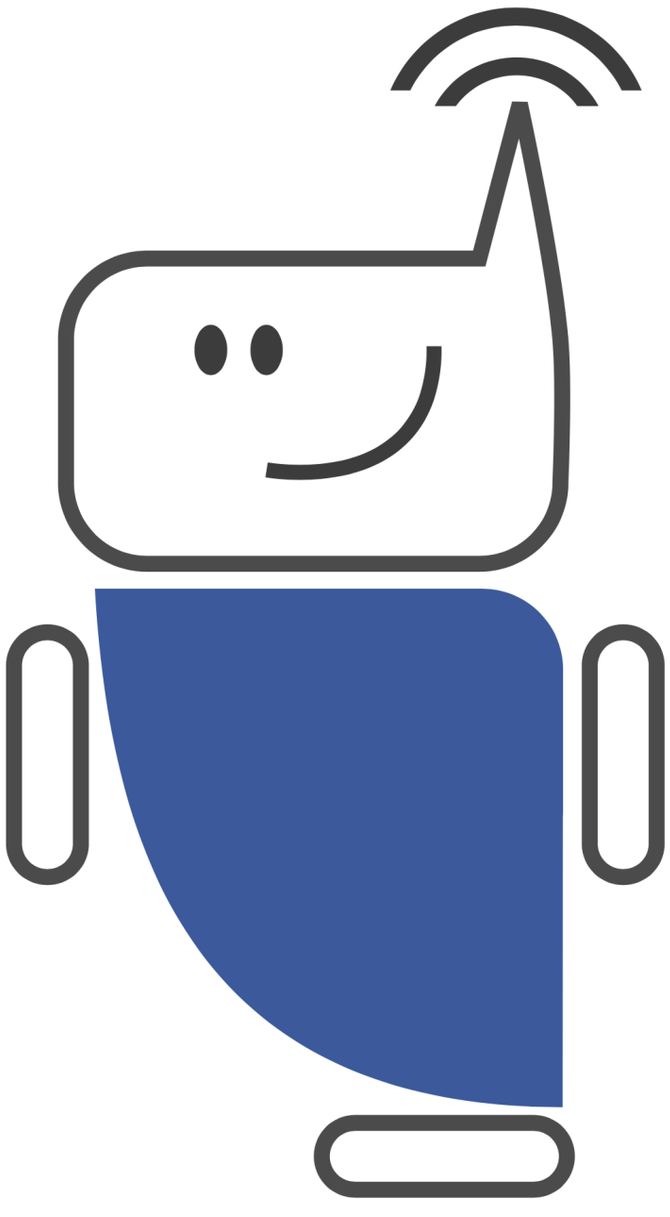
  if (this.salario >= 3000 && this.salario < 4000)
  {
    novoSalario = this.salario + (this.salario * 0.2);
  }

  if (this.salario >= 4000)
  {
    novoSalario = this.salario + (this.salario * 0.1);
  }

  return novoSalario;
}
```

PROBLEMAS

- Além daqueles problemas comentados anteriormente...
- Se o funcionário recebesse menos que $1.000, o fluxo de execução passaria por **todas as estruturas condicionais** seguintes.
- A **repetição das expressões** também é deve ser evitada.
- Muitos passos desnecessários.



REFERÊNCIAS

TERMOS PARA CONSULTA NA LITERATURA DA ÁREA

Os termos não são, necessariamente, sinônimos entre si.

INSTRUÇÕES

COMANDOS

ESTRUTURAS

×

CONDICIONAIS

DE DECISÃO

DE SELEÇÃO

DE CONTROLE

Incluem também as Estruturas de Repetição

\+

DESVIOS CONDICIONAIS / LÓGICOS

INSTRUÇÕES DE DESVIO

REFERÊNCIA

MASCHIO, Eleandro. **Expressões Booleanas e Estrutura Condicional *if-else***. Guarapuava: Universidade Tecnológica Federal do Paraná, 2023. 105 slides, color. Material didático da disciplina de Fundamentos de Programação.

CITAÇÃO COM AUTOR INCLUÍDO NO TEXTO

Maschio (2023)

CITAÇÃO COM AUTOR NÃO INCLUÍDO NO TEXTO

(MASCHIO, 2023)